

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

NOMBRE:
MONCAYO LOOR XAVIER

MATERIA:
APLICACIONES WEB

CARRERA:
INGENIERA DE SOFTWARE

DOCENTE:
GUERRERIO ULLOA GLEISTON



Link de repositorio: <https://github.com/XAML25/Trabajo-formativo-Implementaci-n-comparativa-de-gesti-n-de-estado-en-aplicaciones-web..git>

Gestión de Estado en Aplicaciones Web: Sesiones de Servidor y JSON Web Tokens en PHP, ASP.NET Core y Spring Boot

Resumen— Este informe analiza los mecanismos de gestión de estado utilizados en aplicaciones web sobre el protocolo HTTP, que por definición carece de estado. Se estudian de forma comparativa tres implementaciones de sesiones de servidor —PHP 8 con `$_SESSION`, ASP.NET Core 8 con `ISession` y Java Spring Boot 3 con `HttpSession`— junto con los atributos de seguridad de las cookies (`HttpOnly`, `Secure` y `SameSite`) definidos en el estándar de cookies HTTP. Adicionalmente, se examina el estándar JSON Web Token (JWT) como alternativa de autenticación sin estado (*stateless*), pertinente para arquitecturas de tipo API REST consumidas por aplicaciones de una sola página (SPA). Se concluye que la elección entre sesiones de servidor y JWT depende directamente de la arquitectura de la aplicación: las sesiones son apropiadas para aplicaciones multipágina tradicionales, mientras que JWT resulta más adecuado para APIs REST desacopladas de un frontend SPA.

Palabras clave— sesiones HTTP, cookies, `HttpOnly`, JSON Web Token, JWT, PHP, ASP.NET Core, Spring Boot, seguridad web, REST.

I. INTRODUCCIÓN

El protocolo HTTP fue diseñado como un protocolo sin estado (*stateless*): cada solicitud que un cliente envía a un servidor es procesada de forma independiente, sin que el servidor conserve memoria de interacciones previas [1]. Esta característica, si bien simplifica la arquitectura del protocolo, plantea un problema práctico para aplicaciones web modernas que requieren reconocer a un mismo usuario a lo largo de varias solicitudes, como ocurre en un carrito de compras, un panel autenticado o un flujo de checkout.

Para resolver esta limitación surgieron los mecanismos de gestión de estado, siendo el más extendido el uso de cookies HTTP en conjunto con sesiones de servidor, formalizado en el RFC 6265 [1]. En este esquema, el servidor genera un identificador de sesión (*session ID*), lo entrega al navegador mediante la cabecera `Set-Cookie`, y el navegador lo reenvía automáticamente en cada solicitud posterior a través de la cabecera `Cookie`, permitiendo al servidor recuperar el estado asociado a ese usuario [1].

El presente informe describe, compara y analiza la implementación de este mecanismo en tres plataformas ampliamente utilizadas en el desarrollo web —PHP, ASP.NET Core y Java Spring Boot— y contrasta el enfoque de sesión de servidor con el estándar JSON Web Token (JWT), definido en el RFC 7519 [2], utilizado como alternativa sin estado en arquitecturas de tipo API REST.

II. FUNDAMENTO TEÓRICO

A. El protocolo HTTP y la necesidad de estado

Al ser HTTP un protocolo sin estado, cada par petición-respuesta es independiente entre sí. La especificación de cookies del Internet Engineering Task Force (IETF) resuelve esta limitación definiendo las cabeceras `Set-Cookie` y `Cookie`, que permiten a un servidor almacenar pequeños fragmentos de información —llamados cookies— en el agente de usuario (navegador), y recuperarlos en solicitudes subsiguientes dentro del alcance definido por atributos como `Path`, `Domain` y `Expires` [1].

B. Atributos de seguridad de las cookies de sesión

La guía de gestión de sesiones de OWASP (Open Web Application Security Project) recomienda tres atributos obligatorios para cualquier cookie que transporte un identificador de sesión en producción [3]. El atributo `HttpOnly` impide que scripts del lado del cliente, como JavaScript, accedan a la cookie mediante

document.cookie, mitigando el robo de sesión a través de ataques de Cross-Site Scripting (XSS) [3]. El atributo Secure obliga a que la cookie solo se transmita sobre conexiones cifradas HTTPS, evitando su interceptación en redes no confiables [1], [3]. Finalmente, el atributo SameSite, en su valor Strict, impide que el navegador envíe la cookie en solicitudes originadas desde un sitio distinto al que la emitió, mitigando ataques de tipo Cross-Site Request Forgery (CSRF) [3].

C. JSON Token Web (JWT)

JWT es un estándar abierto, definido en el RFC 7519, que especifica un formato compacto y seguro para URL con el que representar un conjunto de afirmaciones (claims) como un objeto JSON [2]. Dicho objeto puede firmarse digitalmente mediante JSON Web Signature (JWS) o cifrarse mediante JSON Web Encryption (JWE), lo que permite verificar tanto la integridad como, opcionalmente, la confidencialidad de su contenido [2]. A diferencia de una sesión de servidor, un JWT contiene en sí mismo toda la información necesaria para su validación, lo que habilita mecanismos de autenticación sin estado (stateless), donde el servidor no necesita conservar un registro persistente por cada usuario conectado [2].

III. IMPLEMENTACIÓN COMPARATIVA DE SESIONES

8 A. PHP

PHP ofrece soporte de sesiones a través de la extensión nativa Session, que asigna a cada visitante un identificador único (session ID) almacenado por defecto en una cookie [4]. Al invocar session_start(), PHP examina si la solicitud entrante incluye un session ID válido; en caso afirmativo, recupera el entorno previamente guardado en el arreglo superglobal \$_SESSION [4]. La persistencia de los datos de sesión se logra mediante serialización automática gestionada por el propio motor de PHP [4]. El manual oficial de PHP recomienda, además, activar el modo estricto de sesión (session.use_strict_mode) para mitigar ataques de fijación de sesión (session fixation), en los que un atacante induce a la víctima a utilizar un identificador de sesión conocido de antemano [5].

B. ASP.NET Core 8

ASP.NET Core implementa el estado de sesión como un servicio de middleware que debe registrarse explícitamente mediante AddSession() en el contenedor de inyección de dependencias, y habilitarse posteriormente con UseSession() en la canalización de solicitudes [6]. A diferencia de PHP, el objeto ISession solo admite el almacenamiento nativo de cadenas de texto y enteros; por lo tanto, cualquier objeto complejo, como una lista de productos de un carrito de compras, debe serializarse manualmente —habitualmente a JSON— antes de guardarse en la sesión [6]. Por defecto, ASP.NET Core mantiene la sesión inactiva durante 20 minutos antes de expirarla, valor configurable a través de la propiedad IdleTimeout [6].

C. Java Spring Boot 3

En el ecosistema Java, la interfaz HttpSession, definida en la especificación Jakarta Servlet, permite enlazar objetos arbitrarios a la sesión de un usuario mediante el método setAttribute() [7]. Spring Boot obtiene automáticamente una instancia de HttpSession inyectándola como parámetro en los métodos del controlador, sin requerir configuración adicional de middleware como en ASP.NET Core [7], [8]. Cuando el contenedor de servlets necesita serializar la sesión —por ejemplo, para replicarla entre nodos o almacenarla en un backend externo como Redis a través de Spring Session— cualquier clase almacenada en la sesión debe implementar la interfaz Serializable, de lo contrario se produce una excepción NotSerializableException en tiempo de ejecución [8].

D. Cuadro comparativo

La Tabla I resume las diferencias más relevantes entre las tres plataformas evaluadas, con base en la documentación oficial de cada tecnología [1], [4], [6], [7].

TABLA I
Comparación de Mecanismos de Sesión

Criterio	PHP 8	ASP.NET Core 8	Bota de muelle 3
Inicio de sesión	session_start()	AddSession() + UseSession()	HttpSession inyectada por el contenedor
Guardar dato	\$_SESSION['k'] = valor	Session.SetString() (JSON)	session.setAttribute()
Leer dato	\$_SESSION['k']	Sesión.GetString()	session.getAttribute()
Destruir sesión	session_destroy()	Sesión.Clear()	sesión.invalidate()
Cookie por defecto	PHPSESSID	.AspNetCore.Session	JSESSIONID
Timeout por defecto	24 min	20 min	30 min
Almacenamiento	Archivos (/tmp)	Memoria / Redis / SQL	Memoria JVM / Redis / JDBC

IV. JWT FRENTE A SESIONES DE SERVIDOR EN ARQUITECTURAS SPA/REST

Las sesiones de servidor son adecuadas para aplicaciones web tradicionales, de tipo multipágina, en las que el servidor renderiza vistas y mantiene el estado del usuario entre solicitudes sucesivas. Sin embargo, en una arquitectura desacoplada —donde un frontend construido como aplicación de una sola página (SPA) consume una API REST— el uso de sesiones introduce un problema de escalabilidad: el servidor debe mantener estado por cada usuario conectado, lo cual complica el escalado horizontal en despliegues con múltiples instancias, salvo que se recurra a almacenes de sesión distribuidos como Redis [3], [6].

El estándar JWT resuelve este problema porque el propio token, firmado digitalmente, contiene toda la información necesaria para que cualquier instancia del servidor valide al usuario sin necesidad de consultar un almacén de estado compartido, preservando así la naturaleza stateless de las APIs REST [2]. Una práctica recomendada consiste en transportar el JWT (accessToken) en una cookie con el atributo HttpOnly, de modo que JavaScript no pueda acceder a él, reduciendo la superficie de ataque frente a XSS, mientras que un refreshToken de mayor duración —también en cookie HttpOnly— permite renovar el accessToken sin exigir una nueva autenticación completa del usuario [3].

V. DESARROLLO

Class

CarritoController.java

```

package ec.edu.uteq.carrito.controller;

import ec.edu.uteq.carrito.model.Producto;
import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.*;

@Controller
@RequestMapping("/carrito")
public class CarritoController {

    private static final String CLAVE = "carrito";

    private static final Map<String, Double> PRECIOS = Map.of(
        "Laptop", 450.0,
        "Mouse", 25.0,
        "Teclado", 45.0,
        "Monitor", 180.0
    );

    @SuppressWarnings("unchecked")
    private List<Producto> obtenerCarrito(HttpSession session) {
        List<Producto> carrito = (List<Producto>)
session.getAttribute(CLAVE);
        if (carrito == null) {
            carrito = new ArrayList<>();
            session.setAttribute(CLAVE, carrito);
        }
        return carrito;
    }

    @GetMapping
    public String index(HttpSession session, Model model) {
        List<Producto> carrito = obtenerCarrito(session);
        double total = carrito.stream().mapToDouble(Producto::precio).sum();

        model.addAttribute("carrito", carrito);
        model.addAttribute("catalogo", PRECIOS.keySet());
        model.addAttribute("total", total);
        model.addAttribute("sessionId", session.getId());
        return "carrito/index";
    }

    @PostMapping("/agregar")
    public String agregar(@RequestParam String nombre, HttpSession session) {
        if (PRECIOS.containsKey(nombre)) {
            obtenerCarrito(session).add(new Producto(nombre,

```

```

    PRECIOS.get(nombre));
    }
    return "redirect:/carrito";
}

@PostMapping("/eliminar")
public String eliminar(@RequestParam int indice, HttpSession session) {
    List<Producto> carrito = obtenerCarrito(session);
    if (indice >= 0 && indice < carrito.size()) {
        carrito.remove(indice);
    }
    return "redirect:/carrito";
}

@PostMapping("/limpiar")
public String limpiar(HttpSession session) {
    session.removeAttribute(CLAVE);
    return "redirect:/carrito";
}
}

```

Producto.java

```

package ec.edu.uteq.carrito.model;

import java.io.Serializable;

public record Producto(String nombre, double precio) implements Serializable {
    private static final long serialVersionUID = 1L;
}

```

CarritoApplication.java

```

package ec.edu.uteq.carrito;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CarritoApplication {

    public static void main(String[] args) {
        SpringApplication.run(CarritoApplication.class, args);
    }

}

```

Index.html

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Carrito </title>
  <style>
    body{font-family:Arial,sans-serif;max-width:800px;margin:2rem auto}
    table{width:100%;border-collapse:collapse;margin-top:1rem}
    th,td{padding:8px 12px;border:1px solid #ddd;text-align:left}
    th{background:#f49b32;color:white}
    .btn{padding:6px 14px;border:none;border-radius:4px;cursor:pointer}
    .btn-add{background:#27ae60;color:white}
    .btn-del{background:#e74c3c;color:white}
    .btn-clear{background:#7f8c8d;color:white}
  </style>
</head>
<body>
<h1>Tienda - Sesión </h1>

<h2>Catálogo</h2>
<form th:action="@{/carrito/agregar}" method="post">
  <select name="nombre">
    <option th:each="item : ${catalogo}" th:value="${item}"
th:text="${item}"></option>
  </select>
  <button class="btn btn-add">Agregar al carrito</button>
</form>

<h2>Carrito</h2>
<p th:if="${#lists.isEmpty(carrito)}">El carrito está vacío.</p>

<div th:if="${!#lists.isEmpty(carrito)}">
  <table>
    <tr><th>Producto</th><th>Precio</th><th>Acción</th></tr>
    <tr th:each="item, iter : ${carrito}">
      <td th:text="${item.nombre}"></td>
      <td th:text="'$' + ${item.precio}"></td>
      <td>
        <form th:action="@{/carrito/eliminar}" method="post"
style="margin:0">
          <input type="hidden" name="indice"
th:value="${iter.index}">
          <button class="btn btn-del">Eliminar</button>
        </form>
      </td>
    </tr>
  </table>
  <p><strong th:text="'Total: '$ + ${total}"></strong></p>
  <form th:action="@{/carrito/limpiar}" method="post">
    <button class="btn btn-clear">Vaciar carrito</button>
  </form>
</div>

```

```

    </form>
</div>

<p style="color:#999;font-size:0.8rem" th:text="'Session ID: ' +
${sessionId}"></p>
</body>
</html>

```

Application.yaml

```

server:
  port: 8080
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true
        secure: false
        same-site: strict
        name: CARRITO_SESSION

```

Productos agregados

Tienda - Sesión

Catálogo

Mouse Agregar al carrito

Carrito

Producto	Precio	Acción
Mouse	\$25.0	Eliminar
Teclado	\$45.0	Eliminar
Laptop	\$450.0	Eliminar

Total: \$520.0

Vaciar carrito

Session ID: 629183FCC002C99DF96A700FF97BE56

Agregar Producto

Carrito

localhost:8080/carrito

Tienda - Sesión

Catálogo

Mouse

Agregar al carrito

Carrito

Producto	Precio	Acción
Mouse	\$25.0	Eliminar
Teclado	\$45.0	Eliminar
Laptop	\$450.0	Eliminar
Mouse	\$25.0	Eliminar

Total: \$545.0

Vaciar carrito

Session ID: 629183FCC002C99FD96A700FF97BE56

Eliminar Producto

Carrito

localhost:8080/carrito

Tienda - Sesión

Catálogo

Mouse

Agregar al carrito

Carrito

Producto	Precio	Acción
Teclado	\$45.0	Eliminar
Laptop	\$450.0	Eliminar
Mouse	\$25.0	Eliminar

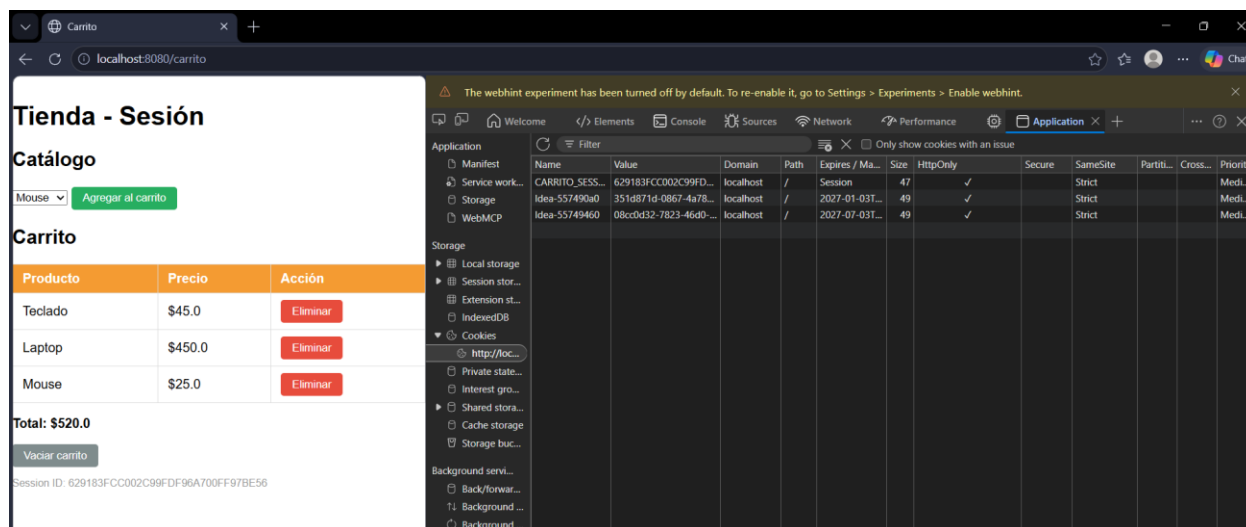
Total: \$520.0

Vaciar carrito

Session ID: 629183FCC002C99FD96A700FF97BE56

Cookie

HttpOnly activo



VI. CONCLUSIONES

El análisis comparativo evidencia que, si bien las tres plataformas estudiadas resuelven el mismo problema —la ausencia de estado en HTTP—, cada una lo aborda con una filosofía distinta: PHP favorece la simplicidad mediante serialización automática; ASP.NET Core prioriza el rendimiento exigiendo serialización explícita; y Spring Boot aprovecha el tipado fuerte de Java a través de la interfaz Serializable. En los tres casos, la protección adecuada del identificador de sesión mediante los atributos HttpOnly, Secure y SameSite —conforme al RFC 6265 y a las recomendaciones de OWASP— resulta indispensable para mitigar ataques de robo de sesión (XSS) y falsificación de solicitudes entre sitios (CSRF).

Finalmente, la decisión entre sesiones de servidor y JWT no debe entenderse como una sustitución universal de una tecnología por otra, sino como una elección arquitectónica: las sesiones continúan siendo la herramienta correcta para aplicaciones multipágina tradicionales, mientras que JWT es la alternativa apropiada para APIs REST consumidas por aplicaciones SPA, por su naturaleza stateless y su compatibilidad con el escalado horizontal.

REFERENCIAS

- [1] A. Barth, "HTTP State Management Mechanism," RFC 6265, IETF, Apr. 2011. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc6265>
- [2] M. Jones, J. Bradley, y N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc7519>
- [3] Fundación OWASP, "Hoja de trucos para la gestión de sesiones", serie de hojas de trucos OWASP. [En línea]. Disponible: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [4] The PHP Group, "Sessions," PHP Manual. [En línea]. Disponible: <https://www.php.net/manual/en/book.session.php>
- [5] The PHP Group, "Session Management Basics," PHP Manual. [En línea]. Disponible: <https://www.php.net/manual/en/features.session.security.management.php>

- [6] Microsoft Corporation, "Gestión de sesiones y estados en ASP.NET Core," Microsoft Learn. [En línea].
Disponible: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state>
- [7] Eclipse Foundation, "HttpSession (Jakarta Servlet API documentation)," Jakarta EE Specifications. [En línea].
Disponible: <https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/http/httpsession>
- [8] VMware, Inc. (Spring Team), "HttpSession Integration," Spring Session Reference Documentation. [En línea].
Disponible: <https://docs.spring.io/spring-session/reference/http-session.html>